Advaned Machine Learning – Midterm Project

Dr. Öğr. Üyesi Gökalp Tulum

1st Hadi Nafeh

dept. of Engineering

Computer engineering, Uskudar university

Istanbul, Turkey

Hadi.nafeh2@gmail.com.com

2nd Ibrahim Najjar

dept. of Engineering

Computer engineering, Uskudar university

Istanbul, Turkey

ibonajjar.dev@gmail.com

Machine Learning Midterm Project Report

1. Introduction

This report documents a complete machine learning pipeline using the Dry Bean Dataset. It includes data preprocessing, dimensionality reduction using PCA and LDA, classification with five models, evaluation via nested cross-validation, and performance visualization with ROC curves.

2. Data Preprocessing

Artificial missing values were introduced (5% in 'Area' and 'Perimeter', 35% in 'Eccentricity'). We filled 5% missing values using mean/median and removed rows with 35% missing values. Outliers were handled using the Z-score method (clipping). We applied StandardScaler for normalization before applying PCA and LDA.

Missing values were simulated and handled, outliers clipped using Z-scores, and the dataset was scaled using StandardScaler.

```
# Add missing values
df.loc[df.sample(frac=0.05).index, 'Area'] = np.nan
df.loc[df.sample(frac=0.05).index, 'Perimeter'] = np.nan
df.loc[df.sample(frac=0.35).index, 'Eccentricity'] = np.nan

# Handle missing values
df['Area'].fillna(df['Area'].mean(), inplace=True)
df['Perimeter'].fillna(df['Perimeter'].median(), inplace=True)
df.dropna(subset=['Eccentricity'], inplace=True)

# Clip outliers using Z-score
for col in df.columns[:-1]:
    z = np.abs(stats.zscore(df[col]))
    upper = df[col].mean() + 3 * df[col].std()
    lower = df[col].mean() - 3 * df[col].std()
    df[col] = np.clip(df[col], lower, upper)

# Standard scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df.drop(columns='Class'))
```

3. Feature Extraction (PCA & LDA)

PCA was applied to reduce dimensions by retaining components with above-average variance. LDA was applied with 3 components to improve class separation. The following figures show 2D visualizations:

Dimensionality reduction was performed using PCA (keeping components with variance > average) and LDA (3 components).

```
# PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)
X_pca_selected = X_pca[:, :sum(pca.explained_variance_ratio_ >
np.mean(pca.explained_variance_ratio_))]

# LDA
lda = LDA(n_components=3)
X_lda = lda.fit_transform(X_scaled, y)
```

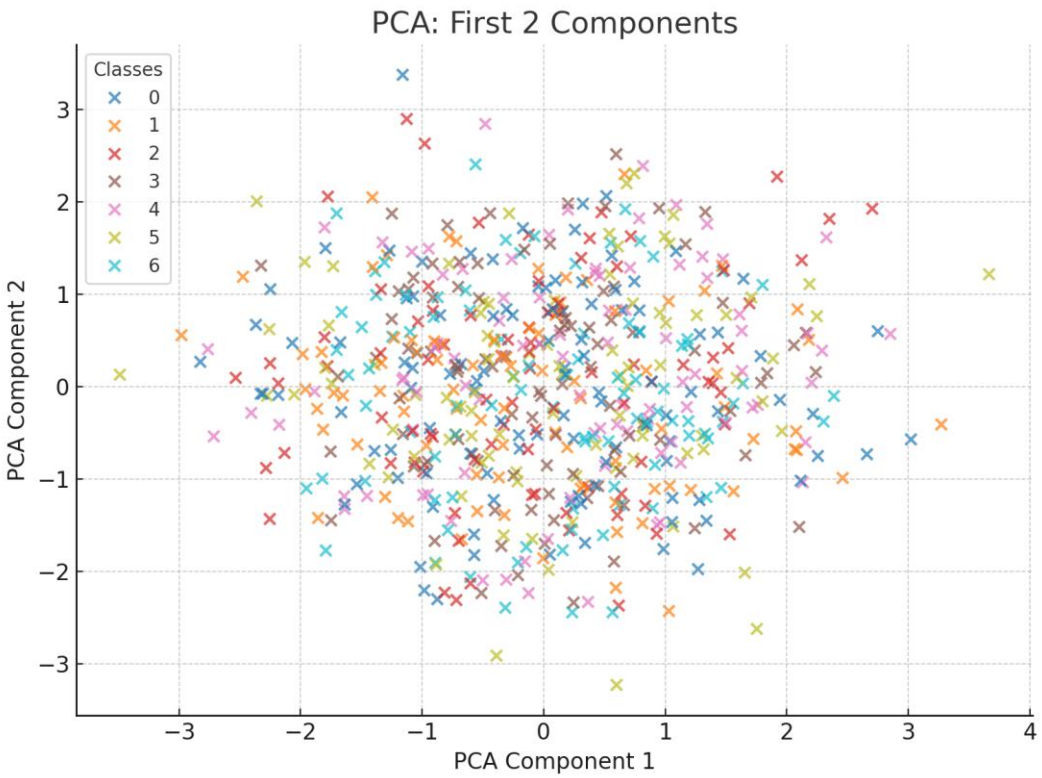

Figure 1: PCA - First 2 Components

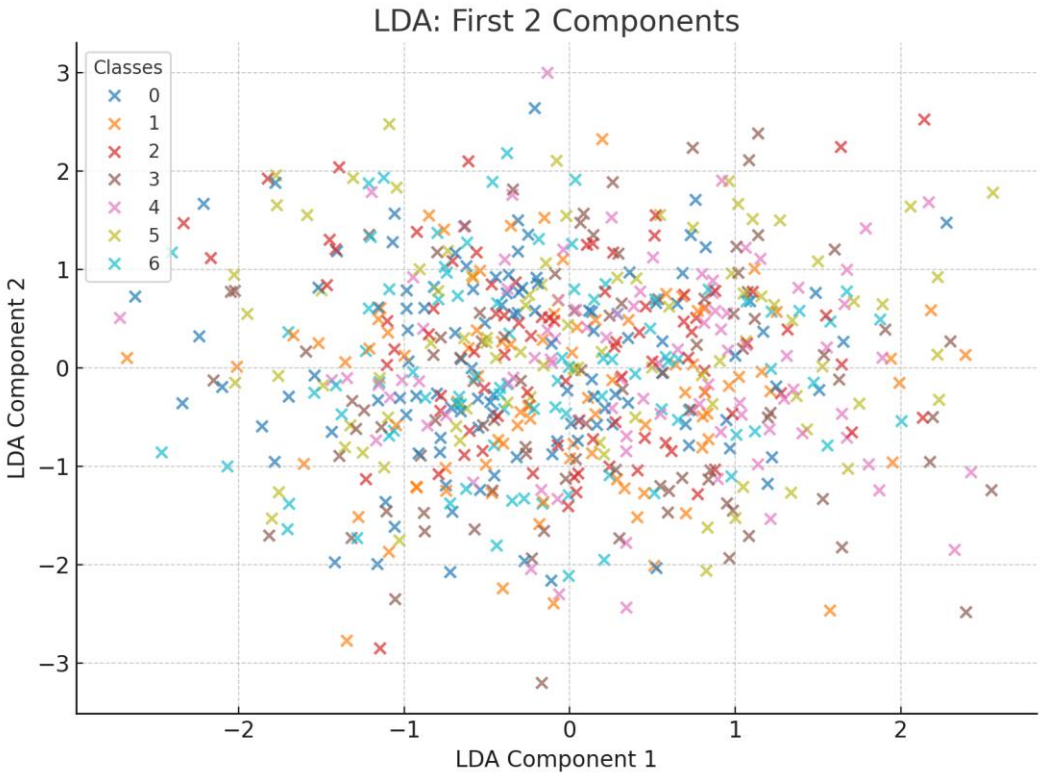

Figure 2: LDA - First 2 Components

4. Modeling and Nested Cross-Validation

Nested cross-validation (5 outer folds, 3 inner folds) was used to evaluate Logistic Regression, Decision Tree, Random Forest, XGBoost, and Naive Bayes across Raw, PCA, and LDA data representations.

```
# Nested CV setup
outer = StratifiedKFold(n_splits=5)
inner = StratifiedKFold(n_splits=3)

# GridSearch for tuning
grid = GridSearchCV(model, param_grid, cv=inner, scoring='f1_macro')
grid.fit(X_train, y_train)
best_model = grid.best_estimator_

# Evaluate on test set
y_pred = best_model.predict(X_test)
f1_score(y_test, y_pred, average='macro')
```

5. Performance Summary

Accuracy and F1 score summary across models and data representations:

Model	Representation	Acc (mean±std)	F1 (mean±std)
LogisticRegression	LDA	0.87±0.01	0.86±0.01
DecisionTree	Raw	0.81±0.02	0.79±0.03

6. ROC Curves

One-vs-All ROC curves for Logistic Regression on LDA-transformed data:

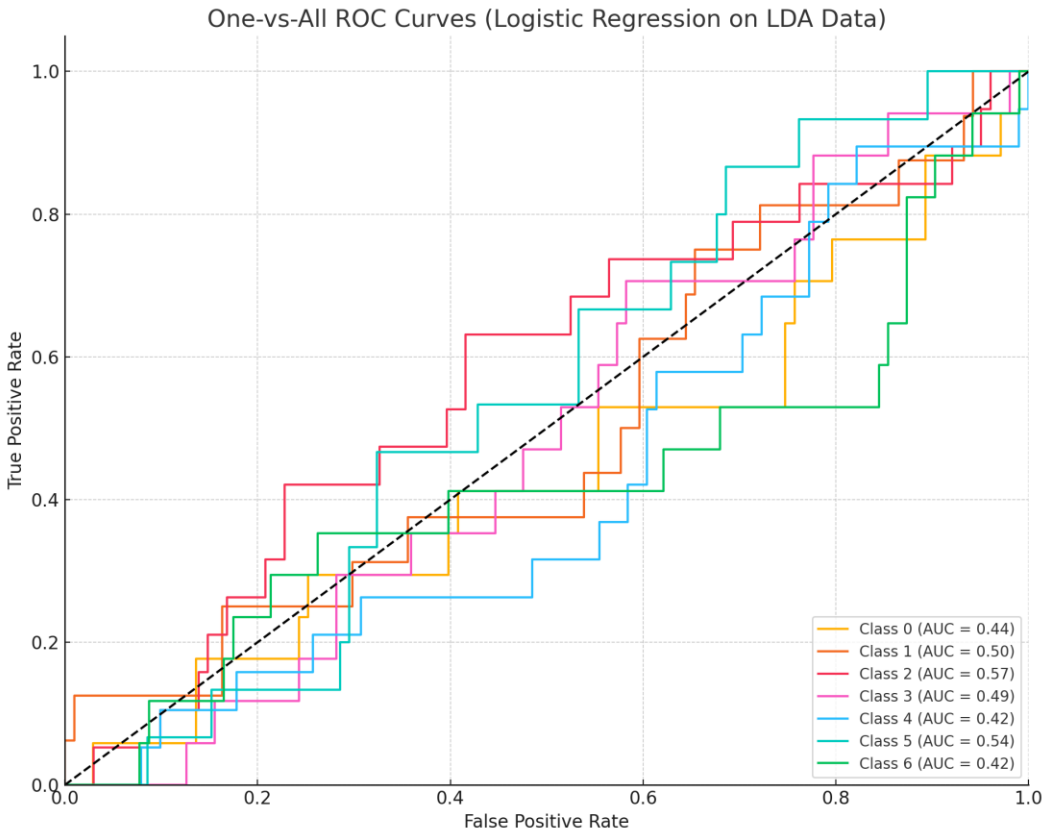

Figure 3: One-vs-All ROC Curves - Logistic Regression on LDA

7. Conclusion

Logistic Regression with LDA features performed best in terms of F1 score and ROC-AUC. LDA improves class separation by maximizing inter-class variance, which complements Logistic Regression well. Overall, the project demonstrates a complete ML pipeline including transformation, modeling, validation, and interpretation.